

Budget Dashboard

Product Requirements Document (PRD) v1.0

Project Name: *BudgetOS*

Version: 1.0

Owner: Ayush Roy

1. Project Vision

The goal is to build a **minimal, fast, and professional personal budget dashboard** that helps users plan and track their monthly spending.

This is **NOT** a finance management application.

This is **NOT** an accounting application.

This is **NOT** an expense management system for businesses.

Instead, it is a **personal budgeting tool** that allows users to:

- Plan their monthly budget
- Allocate money into categories
- Record expenses quickly
- Track remaining balance
- View spending analytics
- Achieve monthly savings goals

The application should feel simple enough to use every day.

2. Core Philosophy

The project follows these principles.

Simplicity over Complexity

Never add a feature that increases complexity without adding real value.

Speed over Features

Adding an expense should take less than **5 seconds**.

Professional Design

The website should look like a premium SaaS product.

It should never look AI-generated.

Consistency

Every page must follow the same spacing, typography, borders, and design language.

User First

Every feature must answer one question:

Does this make managing money easier?

If not, it should not be included.

3. Design Language

The UI should be inspired by clean editorial dashboards.

Design characteristics:

- Cream / warm white background
- Black borders
- Minimal colors
- Professional typography
- Large spacing
- Minimal animations
- Flat cards
- Clean buttons
- Modern layout
- No unnecessary effects

Avoid:

✗ Glassmorphism

- ✗ Neon UI
 - ✗ AI generated gradients
 - ✗ Floating blobs
 - ✗ Fancy loaders
 - ✗ Random icons
 - ✗ Over-animation
-

4. Technology Stack

Frontend

- React
- Vite

Styling

- CSS Modules or Tailwind (decision later)

Charts

- Recharts

Icons

- Lucide React

Database (Phase 3)

- Supabase
- PostgreSQL

Hosting

- Vercel

Version Control

- GitHub
-

5. Pages

The application consists of only a few focused pages.

Dashboard

Daily usage page.

Contains:

- Monthly Allowance
 - Remaining Balance
 - Savings Goal
 - Category Progress
 - Quick Expense Entry
 - Recent Expenses
 - Pie Chart
-

Monthly Setup

Used once every month.

Contains:

- Monthly Allowance
 - Savings Goal
 - Category Allocation
-

Analytics

Contains

- Pie Chart
 - Monthly Spending
 - Weekly Spending
 - Category Reports
-

Settings

Contains

- Manage Categories

- Recurring Expenses
 - Theme (future)
 - Export Data (future)
-

6. Monthly Workflow

Every month begins with:

Step 1

Enter Monthly Allowance

↓

Step 2

Enter Savings Goal

↓

Step 3

Allocate money into categories

↓

Dashboard becomes active

↓

Record expenses during the month

↓

Month ends

↓

Archive

↓

Start new month

7. Expense Workflow

The expense process must always remain extremely simple.

Input Amount

↓

Choose Category

↓

Click Add Expense

↓

Dashboard updates automatically

No notes.

No description.

No receipt.

No unnecessary forms.

8. Categories

Categories are user-defined.

Users can

- Create
- Rename
- Delete
- Allocate Budget
- Change Icon

Nothing should be hardcoded.

9. Savings

Savings are treated as reserved money.

Savings cannot accidentally be spent.

Dashboard always displays

Savings Goal

Current Savings

Remaining Savings Target

10. Analytics

Analytics should answer questions like

Where did my money go?

How much have I spent?

Which category is highest?

Am I within budget?

Nothing more.

11. Performance

The application must

Load quickly

Remain lightweight

Avoid unnecessary packages

Avoid unnecessary API calls

Optimize rendering

12. Mobile Experience

The website must work perfectly on

Desktop

Laptop

Tablet

Phone

The mobile experience is equally important.

13. Database Philosophy

The database stores only essential information.

Never store values that can be calculated.

For example

Store

Allowance

Savings Goal

Expenses

Categories

Calculate

Current Balance

Remaining Budget

Analytics

Progress

Pie Chart

14. Folder Structure

The project must follow a scalable folder structure.

Components

Pages

Hooks

Services

Utils

Assets

Types

Context

Layouts

Charts

15. Coding Standards

Use reusable components.

Avoid duplicate code.

Keep files small.

Meaningful variable names.

Comment only where necessary.

Follow clean architecture.

16. Animations

Animations should be subtle.

Examples

Hover effects

Smooth page transitions

Progress animations

Nothing flashy.

17. Features NOT Included

The following are intentionally excluded.

Loans

Credit Cards

Investments

EMIs

Bank Sync

UPI Integration

AI Chatbot

Tax Reports

Expense Splitting

Social Features

Notifications

These features do not align with the project's philosophy.

18. Future Roadmap

Possible future additions

Dark Mode

Installable PWA

Cloud Backup

Excel Export

PDF Export

Recurring Expenses

Multiple Wallets

Budget Recommendations

19. Success Criteria

The application is successful if:

- ✓ Setting up a new month takes under **2 minutes**.
 - ✓ Adding an expense takes under **5 seconds**.
 - ✓ The Dashboard is understandable within **10 seconds** of opening it.
 - ✓ The UI looks handcrafted, not AI-generated.
 - ✓ The application feels lightweight and enjoyable to use daily.
-

20. Development Strategy

The project will be developed in **three phases**.

Phase 1 — UI & Design

Build the complete interface with mock data, focusing on a polished, responsive, professional design.

Phase 2 — Application Logic

Replace mock data with working application logic using local storage, monthly workflows, category allocation, calculations, and analytics.

Phase 3 — Cloud & Deployment

Integrate Supabase, migrate from local storage to PostgreSQL, prepare the app for deployment on Vercel, optimize performance, and complete production-ready testing.